

Campos dos dados

Guia dos dois arquivos JSON em `data/` , organizado por utilidade: primeiro os campos bons para gráfico, depois os que são só apoio (identificadores, texto) e por fim os que têm problema conhecido e não devem ser usados para comparar grupos.

Contexto rápido: 74 alunos reais, testados em 4 dias (2 turmas de cada uma das 2 instituições), cada aluno usando um de 4 modos de RAG sem saber qual (`none` = baseline sem busca de material, `context` , `self` , `hybrid`). Cada aluno conversa livremente com o chatbot e responde quizzes de lógica de programação.

1. Campos bons para gráfico

Aprendizado — `quiz_answers` (1085 respostas) e `students`

Campo	Onde	O que é	Ideia de gráfico
<code>is_correct</code>	<code>quiz_answers</code>	Se o aluno acertou a pergunta (true/false)	% de acerto por modo de RAG — já feito em <code>quiz/acuracia_por_rag.png</code>
<code>subject</code>	<code>quiz_answers</code>	Assunto da pergunta (Estruturas de Repetição , Vetores , Matrizes e Arrays , linguagem Python)	Cruzar acerto x assunto x modo de RAG — já feito, mas dá pra ver se algum RAG só ajuda num assunto específico
<code>selected_option</code>	<code>quiz_answers</code>	Alternativa escolhida (A–D)	Ainda não usado: dá pra ver se um erro específico (distrator) se repete mais num modo de RAG que em outro
<code>total_correct_answers</code> / <code>total_wrong_answers</code>	<code>students</code>	Total de acertos/erros por aluno	Distribuição de acerto por aluno dentro de cada modo (não só a média)

Engajamento — `students`

Campo	O que é	Ideia de gráfico
<code>active_chat_time_sec</code>	Tempo ativo de chat, calculado a partir dos horários reais das mensagens (ver ressalva na seção 3 sobre por que não é <code>total_usage_time_sec</code>)	Tempo médio de sessão por modo — já feito
<code>user_message_count</code>	Nº de mensagens que o aluno enviou	Mensagens médias por aluno por modo — já feito
<code>chat_session_count</code>	Nº de conversas distintas que o aluno abriu	Ainda não usado: alunos abrem mais conversas novas

Campo	O que é	Ideia de gráfico
		em algum modo, ou preferem continuar a mesma?
<code>topics</code>	Quantas mensagens o aluno mandou sobre cada assunto de programação (objeto tópico → contagem)	Ainda não usado: quais tópicos os alunos mais perguntam, e se isso muda por modo de RAG

Comportamento do RAG — `chat_message_metrics` (511 respostas do assistente)

Campo	O que é	Ideia de gráfico
<code>retrieved_chunk_count</code>	Quantos trechos de material o RAG recuperou para montar a resposta	Taxa de recuperação (% de respostas com chunk > 0) por modo — já feito
<code>avg_retrieval_score</code> / <code>max_retrieval_score</code>	Pontuação de relevância dos trechos recuperados (quanto maior, mais o trecho bate com a pergunta)	Score médio por modo — já feito; dá pra ver também a distribuição (não só a média)
<code>timings_ms.total</code>	Tempo total de resposta da IA, em milissegundos	Tempo médio de resposta por modo — já feito
<code>tokens.total_tokens</code>	Tokens totais gastos na resposta (prompt + completion)	Tokens médios por modo — já feito
<code>tokens.prompt_tokens</code> vs <code>tokens.completion_tokens</code>	Quanto do texto é o material injetado pelo RAG (prompt) vs a resposta gerada (completion)	Ainda não usado: mostra o "custo" de contexto que cada modo de RAG adiciona — hybrid/context tendem a ter prompt bem maior que none
<code>detected_topic</code>	Tópico da pergunta, detectado automaticamente	Cruzar qualidade de recuperação (<code>avg_retrieval_score</code>) por tópico — mostra se o RAG recupera bem só em alguns assuntos

2. Campos de apoio (identificadores e texto — não servem para gráfico direto)

Esses campos existem para relacionar as tabelas entre si ou para leitura qualitativa, não para plotar:

- `student_id`, `message_id`, `chat_session_id` — identificadores únicos, usados só para juntar as tabelas (ex.: achar todas as mensagens de um aluno).
- `name`, `email` — identificação do aluno (nome já está anonimizado como "Participante NN"). Sem uso analítico.
- `university`, `day`, `class_name`, `group`, `rag_mode` — não são "métricas", são os filtros/eixos usados para comparar os grupos nos gráficos acima (ex.: agrupar por `rag_mode`, separar por `day`).
- `course_name` — nome do curso na instituição de origem. É redundante com `university` + `class_name` e a formatação é inconsistente entre instituições (um lugar usa nome, "Segundo Ano Técnico", outro usa código de disciplina, "I2261TI") — melhor não usar para gráfico.

- `content` (mensagens) e `retrieved_chunks[].content` (mensagens) — texto completo da pergunta/resposta e dos trechos recuperados. Só em `logibot-chat-messages.json`. Útil para ler exemplos reais e ilustrar um gráfico com uma citação, não para plotar.
 - `timings_ms.agent`, `db/history`, `rag/settings`, `embedding`, `lexical query`, `chroma query`, `eligible documents`, `agent/self decision`, `agent/self final` — sub-etapas internas do tempo de resposta (debug de engenharia). Só `timings_ms.total` é usado nos gráficos; o resto é detalhe técnico demais para uma comparação pedagógica, mas fica disponível se quiser abrir o "gargalo" do tempo de resposta.
 - `provider` — motor que gerou a resposta (`digitalocean-agent` ou `self-rag:digitalocean-agent`). Na prática só confirma se o modo era `self` ou não — informação que `rag_mode` já dá. Redundante.
-

3. Cuidado: campos com problema conhecido

- `students[].total_usage_time_sec` — vem quebrado na origem: está zerado em 43 dos 74 alunos ativos (58%), inclusive em contas com chat e quiz reais registrados. É falha de instrumentação do próprio app, não da extração. **Não usar para comparar grupos.** Fica no JSON só por transparência; os gráficos usam `active_chat_time_sec` no lugar (calculado a partir dos timestamps reais das mensagens — só disponível para os 60 alunos que têm chat).
 - `students[].level` — hoje tem um único valor (`iniciante`) em todos os 74 alunos. Sem variação, não dá para comparar nada com esse campo (a menos que uma turma futura tenha alunos de nível diferente).
-

Outras fontes

- `logibot-db.sql` — dump bruto original do banco de dados (não versionado no git), de onde os dois JSONs foram extraídos. Só necessário se for preciso ver a estrutura original das tabelas.